SEHAT-SETU -- System Design

BY- Akash kumar(25LBCS3010) & Somanshu Kumar(25LBCS3033)

1. Project purpose

SEHAT-SETU is a healthcare referral analytics and gap-identification platform. It analyzes referral records alongside facility capacity information to help health administrators identify possible shortages in specialists, diagnostic services, equipment, beds, and other resources. It is not merely a system for entering or tracking referrals.

2. Problem statement

Healthcare facilities refer patients when a required service, specialist, diagnostic test, equipment, or capacity is unavailable or insufficient. If referral data remains isolated, district authorities may struggle to see repeated patterns across facilities. SEHAT-SETU combines referral trends with facility service/capacity data to highlight potential gaps that need investigation and planning.

3. Scope for the student demo

- Enter and view sample referrals with source facility, destination facility, date, reason, required service/specialty and status.
- Maintain a list of facilities, their service availability, specialist counts, and selected resource/equipment availability.
- Show dashboard summaries by district, facility, service, date and referral reason.
- Flag possible gaps when repeated referrals for a service come from facilities that report that service or capacity as unavailable.
- Provide role-based access and record important user actions.
- Use dummy or public data only; do not use identifiable real patient records.

4. Keep the first version simple

Build a modular monolith: one React frontend, one FastAPI backend and one Azure SQL database. Use normal Python/Pandas aggregation and simple, explainable rules first. Scikit-learn is optional and should only be added after the core dashboard works. ABDM/FHIR is a future integration, not a blocker for the demo.

5. Users and roles

Role	What they can do
Facility staff	Enter referrals and update their facility's capacity information.
District officer	View district-wide referral trends and potential gap reports.
State officer	View State-wide referral trends and potential gap reports. And supervise district level

Administrator	Manage users, facilities, services and reference data.

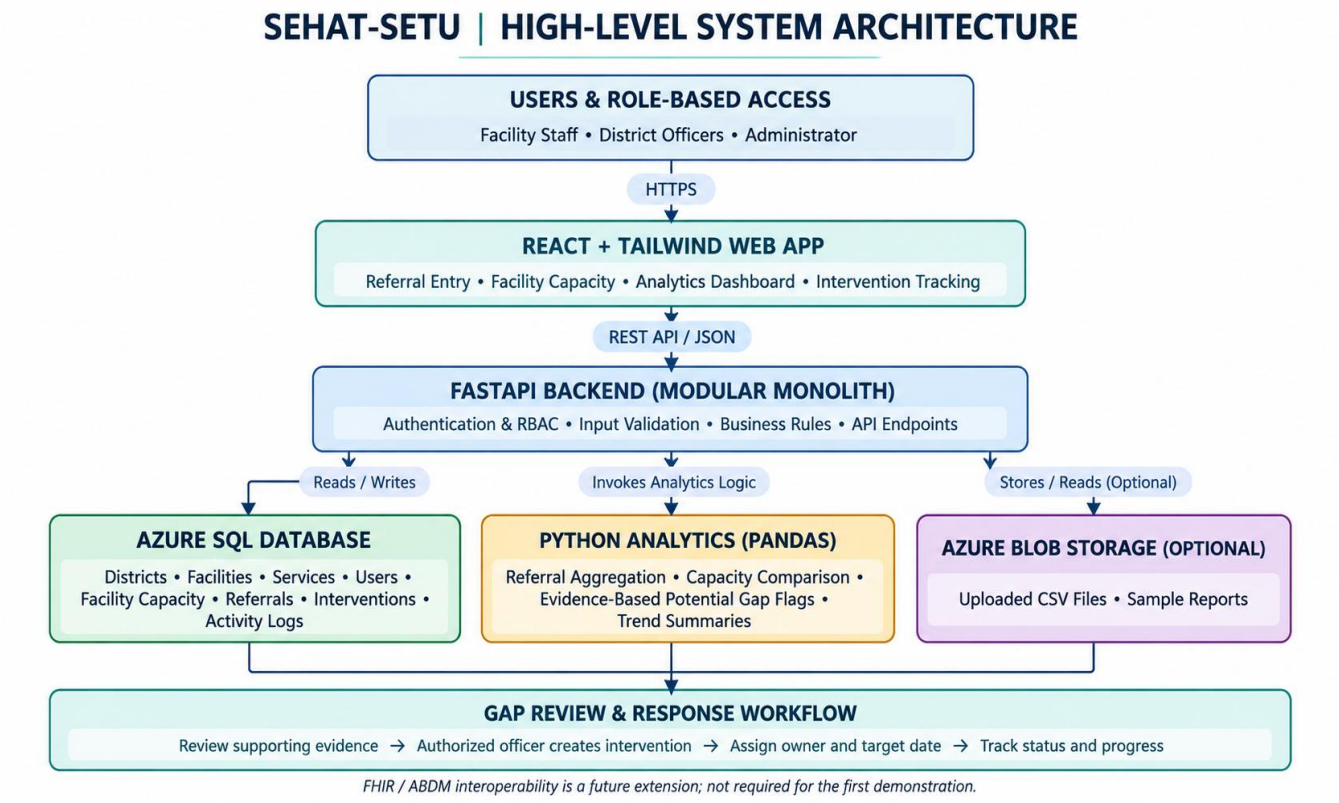
6. Functional requirements

- FR1: Users can log in and access features permitted by their role.
- FR2: Authorized users can create and view referrals.
- FR3: Administrators can maintain facility, service and capacity data.
- FR4: The dashboard can group referral counts by district, facility, service, reason and time period.
- FR5: The system can compare referral patterns with reported service/capacity data and flag possible gaps.
- FR6: Users can view the evidence behind a flag, such as referral count and reported service availability.
- FR7: The system records key actions in an activity log.

7. Non-functional requirements

- Security: HTTPS, hashed passwords, role checks, parameterized SQL and secrets kept outside source control.
- Privacy: use synthetic or public data for the student demonstration.
- Usability: simple filters, readable charts and clear explanations for each potential gap.
- Reliability: validate input and handle missing or inconsistent data without crashing.
- Maintainability: keep the first release in one backend application with clearly separated modules.

8. High-level architecture



9. API list (endpoints provided by our FastAPI backend)

Method	Endpoint	Purpose
POST	`/api/auth/login`	Authenticate a user.
GET	`/api/facilities`	List facilities and district information.
GET	`/api/services`	List service/specialty categories.
GET	`/api/facility-capacity`	View reported services, specialist counts and resource availability.
PUT	`/api/facility-capacity/{id}`	Update a facility's capacity information.
GET	`/api/referrals`	List referrals; support date, district, facility, service and reason filters.
POST	`/api/referrals`	Create a referral record.
GET	`/api/analytics/summary`	Return referral totals and breakdowns.
GET	`/api/analytics/gaps`	Return possible gaps supported by referral and capacity data.
GET	`/api/analytics/patterns`	Optional unusual-pattern flags, if the ML feature is implemented.
GET	`/api/activity-logs`	View permitted activity logs.

These are application API endpoints built with FastAPI. ABDM endpoints are external government health-system APIs; they are not included as working endpoints in the first demo unless access and authorization are available.

10. Technology stack

Technology	Use
React + Tailwind CSS	Frontend and dashboard.
Python + FastAPI	Backend API and access-control checks.
Azure SQL Database	Relational data store.
Python + Pandas	Data cleaning, grouping and basic analytics.
Scikit-learn	Detect unusual patterns after baseline analytics work.
Azure Blob Storage (optional)	Store uploaded sample CSV/report files.
Power BI (optional)	Advanced reporting; built-in charts are enough for the first version.
FHIR / ABDM (future)	Interoperability extension, subject to supported APIs, sandbox access and authorization.

11. System design trade-off

Trade-off: Monolithic Architecture vs. Microservices Architecture

A monolithic architecture keeps the application's modules, such as authentication, referral management, facility capacity management, and analytics, within a single backend application. It is easier to develop, test, deploy, and maintain, making it suitable for the initial version of SEHAT-SETU. However, as the application grows, maintaining and scaling individual components independently may become difficult.

A microservices architecture separates these modules into independent services that can be developed, deployed, and scaled separately. However, it introduces additional complexity in communication, deployment, monitoring, and security.

Decision: SEHAT-SETU will use a **modular monolithic architecture** with FastAPI for the initial release because it is simpler, cost-effective, and suitable for project. Microservices may be considered in the future if the platform grows and requires independent scaling of its components.

12. Threat Model

Threat	Possible Impact	Simple Mitigation

Unauthorized access	Someone accesses restricted data or features.	Implement login and role-based access control (RBAC).
SQL injection	Malicious input manipulates database queries.	Use parameterized SQL queries instead of building queries by concatenating user input.
Weak password storage	Stolen database records expose user passwords.	Store password hashes using bcrypt or Argon2, never plain-text passwords.
API key exposure	Someone misuses cloud or AI services.	Keep API keys in environment variables and add .env to .gitignore.
Invalid user input	Incorrect or unexpected data causes errors or corrupts records.	Validate inputs on the backend using FastAPI and Pydantic.
Cross-site scripting (XSS)	Malicious scripts may execute in the browser.	Use React's default text rendering and avoid unsafe HTML injection such as dangerouslySetInnerHTML.

13. Design review checklist

- The project goal is referral analysis and healthcare gap identification, not just referral CRUD.
- The database includes both referrals and facility capacity/service availability.
- The dashboard includes filters and evidence for each possible gap.
- The first version can run without ABDM integration or ML.